Project Report

Efficient Transformer Inference for Text Classification

DistilBERT · SST-2 · Dynamic Quantization · torch.compile

Course: CSCI 611
Team Members: Abinesh, Akshith, Arushi
GitHub Repository: https://github.com/amacharla15/CSCI611_GROUP7/tree/main/FinalProject
Final Project Folder: DistilBERT-TextClassification_Quantization_CompiledExecution/efficient-transformer-inference

---

1. Overall Context and Motivation

Transformer-based natural language processing models are widely used for text classification tasks such as sentiment analysis, intent detection, spam filtering, toxicity detection, document tagging, and customer support ticket routing. These models can achieve strong predictive performance, but deployment requires more than accuracy alone. In real systems, a model must also respond quickly, use memory efficiently, and maintain reliable predictions after optimization.

This project studies the tradeoff between **model quality** and **inference efficiency** for a transformer text classification model. We used **DistilBERT** as the base model and the **SST-2 sentiment classification dataset** as the task. The final model was fine-tuned on the full SST-2 training split and then optimized using two inference-focused methods:

1. **Dynamic INT8 quantization**
2. **Compiled execution using torch.compile**

The goal was not only to make the model faster, but also to measure whether optimization changed accuracy, model size, throughput, memory usage, or individual predictions.

The final FP32 baseline achieved **90.48% validation accuracy**, or **789 correct predictions out of 872 validation examples**. Dynamic quantization reduced model size by **48.2%** and improved batch-8 latency by **1.83×**, but caused a small accuracy drop. torch.compile preserved accuracy and gave a modest steady-state latency improvement, but had high first-call compilation overhead.

---

2. Relevant Work and Background

2.1 Transformer Models

Transformer models use self-attention to learn relationships between tokens in a sequence. Unlike older sequence models that process text mostly step-by-step, transformers allow each token to attend to other tokens in the sentence. This is useful for sentiment classification because words like “bad,” “excellent,” or “not” must be interpreted in context.

For example, in the sentence:

this movie is bad

the model should learn that “bad” strongly affects the sentiment of “movie.” Attention helps the model represent these relationships.

2.2 DistilBERT

DistilBERT is a smaller and faster version of BERT. It keeps much of the language understanding capability of BERT while reducing model size and inference cost. Because our project focuses on inference efficiency, DistilBERT is a good baseline: it is a real transformer model, but small enough to fine-tune and benchmark within the project timeline.

2.3 SST-2 Sentiment Classification

SST-2 is a binary sentiment classification dataset. Each example contains a sentence and a label:

0 = negative
1 = positive

For the final project, we used:

Training split: 67,349 examples
Validation split: 872 examples

We report **validation accuracy**, not test accuracy, because the SST-2 validation split is the labeled split used for local evaluation.

2.4 Quantization

Quantization reduces the numerical precision of model parameters. In our project, dynamic quantization converted mainly Linear layer weights from FP32 representation into INT8 representation with scale metadata. This reduces model size and can speed up CPU inference.

A simplified example:

FP32 weight: 0.123456
scale: 0.01

INT8 stored value: round(0.123456 / 0.01) = 12
approximate represented value: 12 × 0.01 = 0.12

The model does not treat 12 as the real weight directly. It uses the INT8 value together with scale information. This compression saves memory, but introduces small rounding differences that can slightly change logits and predictions.

2.5 torch.compile

torch.compile changes the runtime execution path of a PyTorch model. It does not compress the model weights. Instead, PyTorch attempts to capture model operations into graph regions using TorchDynamo, then compile and optimize those graph regions.

In our project, torch.compile preserved model accuracy and slightly improved steady-state CPU latency, but the first compiled call had high overhead.

---

3. High-Level Framework of the Solution

Our solution followed this pipeline:

Raw SST-2 sentence
↓
Tokenizer
↓
input_ids + attention_mask
↓
Fine-tuned DistilBERT FP32 checkpoint
↓
Three inference variants:
1. Eager FP32 baseline
2. Dynamic INT8 quantized model
3. Compiled FP32 model using torch.compile
↓
Benchmark and compare:
accuracy
correct count
latency
throughput
model size
memory

compile overhead
prediction changes

3.1 Uniqueness of Our Solution

The uniqueness of this project is that we did not only report model accuracy. We built a practical inference optimization study with multiple deployment-style metrics.

The project includes:

- Full SST-2 fine-tuning on **67,349 training examples**
- A stable FP32 reference checkpoint
- Dynamic INT8 quantization
- torch.compile runtime optimization
- Accuracy, latency, throughput, model size, memory, and compile overhead measurement
- Prediction-level diagnostics showing how quantization changed individual predictions
- Profiler evidence showing what torch.compile actually changed
- Honest reporting of what worked and what did not work

This makes the project more than a basic classification model. It is an experimental study of model deployment tradeoffs.

---

4. Detailed Solution and Implementation

4.1 Dataset Loading and Tokenization

The SST-2 dataset contains raw English sentences. DistilBERT cannot process raw strings directly, so we tokenize each sentence into numerical tensors.

The tokenizer creates:

input_ids: token IDs from the vocabulary
attention_mask: 1 for real tokens, 0 for padding
labels: ground-truth class labels

Important tensor shapes:

input_ids: [B, L]
attention_mask: [B, L]
labels: [B]

where B is batch size and L is sequence length.

Code Snippet: Tokenization

```
def tokenize_function(examples):
    return tokenizer(
        examples["sentence"],
        truncation=True,
        padding="max_length",
        max_length=128
    )
```

This converts each sentence into fixed-size input tensors so examples can be processed in batches.

4.2 FP32 Baseline Fine-Tuning

The baseline model was created from:

distilbert-base-uncased

We fine-tuned it for sequence classification with two output labels:

0 = negative
1 = positive

Training setup:

Base model: distilbert-base-uncased
Training examples: 67,349
Validation examples: 872
Epochs: 1
Seed: 42
Training hardware: A100 GPU
Evaluation metric: validation accuracy

The final FP32 checkpoint achieved:

Validation accuracy: 90.48%
Correct predictions: 789 / 872

Code Snippet: Loading Model

```
model = AutoModelForSequenceClassification.from_pretrained(
   "distilbert-base-uncased",
   num_labels=2
)
```

Code Snippet: Training Structure

```
model.train()

for epoch in range(num_epochs):
   for batch in train_loader:
      optimizer.zero_grad()

      outputs = model(
         input_ids=batch["input_ids"].to(device),
         attention_mask=batch["attention_mask"].to(device),
         labels=batch["labels"].to(device)
      )

      loss = outputs.loss
      loss.backward()
      optimizer.step()
```

The model was trained for one full epoch over the full SST-2 training split. One epoch was acceptable because this was fine-tuning a pretrained transformer, not training a language model from scratch.

4.3 Checkpoint Saving

After fine-tuning, the trained model was saved as a checkpoint.

A checkpoint is the saved trained model folder. It includes the learned weights, model configuration, tokenizer files, and training summary.

Final checkpoint:

checkpoints/our_finetuned_distilbert_sst2_full_seed42

Important files:

model.safetensors
config.json
tokenizer.json
tokenizer_config.json
training_summary.json

This checkpoint was then reused for all inference variants so that FP32, INT8, and compiled FP32 were compared fairly.

4.4 Dynamic INT8 Quantization

Dynamic quantization was applied after training. It did not retrain the model. It loaded the trained FP32 checkpoint and converted mainly Linear layers to quantized versions.

A Linear layer performs:

output = input × weight + bias

DistilBERT contains many Linear layers in:

query projection
key projection
value projection
attention output projection
feed-forward layers
classifier head

Because transformer inference spends significant time in Linear-layer matrix multiplication, applying INT8 quantization to these layers can reduce model size and improve CPU inference speed.

Code Snippet: Dynamic Quantization

```
from torch.ao.quantization import quantize_dynamic

int8_model = quantize_dynamic(
    fp32_model,
    {torch.nn.Linear},
    dtype=torch.qint8
)
```

This produced a smaller model:

FP32 size: 255.45 MB
INT8 size: 132.29 MB

However, quantization only approximates FP32 values, so predictions can change slightly.

4.5 torch.compile Optimization

torch.compile was applied to the same trained FP32 checkpoint. Unlike quantization, it does not shrink the stored model weights. It changes execution.

Code Snippet: Compiled Model

```
compiled_model = torch.compile(fp32_model)
```

The first call triggers compilation:

TorchDynamo captures graph
backend compiles graph region
compiled execution is reused for repeated calls

For our full model, torch.compile preserved accuracy but had high first-call overhead.

Final local batch-8 result:

FP32 baseline latency: 435.28 ms
Compiled FP32 latency: 394.05 ms
First compiled call: 81.52 s

4.6 Benchmarking Methodology

For fair comparison, all variants used the same trained checkpoint and the same validation split.

Benchmark settings:

Validation examples: 872
Final comparison batch size: 8
Warmup steps: 5
Measured steps: 20
Inference device: local CPU
Training device: A100 GPU

We used CPU inference benchmarking because PyTorch dynamic quantization mainly targets CPU Linear layer inference.

Code Snippet: Evaluation Mode

```
model.eval()

with torch.no_grad():
   outputs = model(
      input_ids=input_ids,
      attention_mask=attention_mask
   )
```

model.eval() disables training behavior such as dropout. torch.no_grad() avoids gradient tracking because inference does not update weights.

Code Snippet: Latency and Throughput

```
start = time.perf_counter()

with torch.no_grad():
   outputs = model(
      input_ids=input_ids,
      attention_mask=attention_mask
   )

end = time.perf_counter()

latency_seconds = end - start
throughput = batch_size / latency_seconds
```

Throughput means how many samples the model processes per second.

For example:

batch size = 8
latency = 0.43528 seconds
throughput = 8 / 0.43528 = 18.38 samples/sec

4.7 Prediction-Level Diagnostic

Accuracy alone does not show whether optimized models make the same predictions. Therefore, we compared FP32 and INT8 predictions example by example.

Code Snippet: Prediction Flip Analysis

```
fp32_pred = torch.argmax(fp32_logits, dim=1)
int8_pred = torch.argmax(int8_logits, dim=1)

if fp32_pred != int8_pred:
   flipped_predictions += 1

if fp32_pred == label and int8_pred != label:
   fp32_correct_int8_wrong += 1

if fp32_pred != label and int8_pred == label:
   fp32_wrong_int8_correct += 1
```

A diagnostic run showed that quantization changed some predictions, including both harmful and helpful flips. This confirmed that the INT8 model was not prediction-identical to FP32.

4.8 torch.compile Graph and Profiler Inspection

To avoid saying only that torch.compile “optimized something,” we inspected what happened.

TorchDynamo graph capture showed:

Graph count: 1
Graph breaks: 0
Captured operations: 102

Captured operation families included:

embedding
linear
scaled_dot_product_attention
layer_norm
GELU
dropout
classifier linear

We also used PyTorch profiler to compare eager FP32 and compiled FP32.

Profiler comparison:

Metric / Operation	Eager FP32	Compiled FP32	Change
Total self CPU time	4.606 s	3.796 s	17.6% lower
aten::copy_	325.20 ms	168.07 ms	48.3% lower
scaled-dot-product attention	227.23 ms	140.39 ms	38.2% lower
Batch-8 latency	435.28 ms	394.05 ms	9.5% lower

The profiler also showed CompiledFxGraph and Torch-Compiled Region, confirming compiled execution. However, aten::addmm, which corresponds to Linear-layer matrix multiplication, remained the dominant cost. Therefore, torch.compile reduced overhead but did not remove the main transformer compute bottleneck.

---

5. Test Results and Analysis

5.1 Final Batch-8 Results

The final comparison used the full-SST-2 trained checkpoint and local CPU batch-8 inference.

Variant	Accuracy	Correct / 872	Latency	Speedup	Throughput	Model Size	First Call Compile
FP32 baseline	0.9048	789	435.28 ms	1.00×	18.38/s	255.45 MB	0.00 s
Dynamic INT8	0.8968	782	238.40 ms	1.83×	33.56/s	132.29 MB	0.00 s
Compiled FP32	0.9048	789	394.05 ms	1.10×	20.30/s	255.45 MB	81.52 s

5.2 Accuracy Analysis

The FP32 baseline achieved:

90.48% validation accuracy
789 / 872 correct

The INT8 model achieved:

89.68% validation accuracy
782 / 872 correct

This is a small but real accuracy drop:

90.48% - 89.68% = 0.80 percentage points

Compiled FP32 preserved the FP32 baseline accuracy:

90.48%
789 / 872 correct

Therefore, quantization improved efficiency but slightly reduced accuracy, while compile preserved accuracy.

5.3 Latency Analysis

Latency measures how long one batch takes.

Batch-8 latency:

FP32 baseline: 435.28 ms
Dynamic INT8: 238.40 ms
Compiled FP32: 394.05 ms

Dynamic INT8 was the fastest:

435.28 ms → 238.40 ms
1.83× speedup

Compiled FP32 gave a smaller improvement:

435.28 ms → 394.05 ms
1.10× speedup

5.4 Throughput Analysis

Throughput measures how many samples are processed per second.

Batch-8 throughput:

FP32 baseline: 18.38 samples/sec
Dynamic INT8: 33.56 samples/sec
Compiled FP32: 20.30 samples/sec

Dynamic INT8 had the best throughput because it processed more examples per second.

5.5 Model Size Analysis

Model size:

FP32 baseline: 255.45 MB
Dynamic INT8: 132.29 MB
Compiled FP32: 255.45 MB

Dynamic quantization reduced model size by:

255.45 MB → 132.29 MB
48.2% reduction

Compiled FP32 did not reduce model size because torch.compile changes execution, not stored weights.

5.6 Compile Overhead Analysis

torch.compile had a high first-call cost:

First compiled call: 81.52 seconds

This happens because the first call must capture and compile the model graph. After compilation, repeated calls can reuse the compiled graph.

This means compiled execution is more useful for long-running inference services than for short scripts that only run a few predictions.

---

6. Comparison with Earlier Group Results

During development, we initially used a smaller 4,000-example training subset to debug the full pipeline. That early version was useful for development because it allowed fast iteration, but it was not appropriate as the final result after full-dataset training became available.

6.1 Earlier Development Run

Preliminary subset run:

Training examples: 4,000
Validation examples: 872
FP32 accuracy: 87.50%

This helped us test:

training script
benchmarking script
dynamic quantization
torch.compile
result logging
figure generation

However, it was not our final result because it did not use the full SST-2 training split.

6.2 Final Full-Dataset Run

Final run:

Training examples: 67,349
Validation examples: 872
FP32 accuracy: 90.48%

The final full-dataset model is better aligned with the project proposal and gives a stronger baseline for evaluating inference optimization.

---

7. Comparison with Reference/Baseline Solutions

Our main reference solution is the unoptimized **FP32 DistilBERT baseline**. All optimization results are compared against this baseline.

The FP32 baseline represents the standard approach:

fine-tune pretrained DistilBERT
run normal eager FP32 inference
measure validation accuracy and latency

Compared with this reference:

- Dynamic INT8 reduced latency and model size, but caused a small accuracy drop.
- Compiled FP32 preserved accuracy and slightly reduced latency, but did not reduce model size and had high compile overhead.

We also compared our implementation approach with common transformer deployment ideas:

standard pretrained transformer fine-tuning
dynamic quantization for CPU inference
runtime compilation using torch.compile
profiler-based performance inspection

Our results support the common deployment idea that optimization must be evaluated on multiple metrics, not only accuracy.

---

8. What Worked vs. What Did Not Work

8.1 What Worked

Full SST-2 Fine-Tuning Worked

Fine-tuning on the full SST-2 training split worked successfully using the A100 GPU. The model reached:

90.48% validation accuracy
789 / 872 correct

Dynamic Quantization Worked

Dynamic quantization successfully reduced model size:

255.45 MB → 132.29 MB

It also improved batch-8 CPU latency:

435.28 ms → 238.40 ms

torch.compile Worked Locally

torch.compile worked in the local WSL environment and preserved accuracy:

FP32: 90.48%
Compiled FP32: 90.48%

It improved steady-state batch-8 latency:

435.28 ms → 394.05 ms

Profiling Worked

PyTorch profiler showed that compiled execution used a compiled graph region and reduced some runtime overhead. It also showed that Linear-layer matrix multiplication remained the main bottleneck.

8.2 What Did Not Work

Early 4,000-Example Run Was Not Final

The early subset experiment was useful for debugging, but it was not strong enough for final reporting. We replaced it with the full 67,349-example training run.

torch.compile Failed on the University Server

On the university GPU server, torch.compile failed because TorchInductor required Python.h, and the environment did not have Python development headers installed. This was an environment/toolchain issue, not a model failure.

We solved this by transferring the full trained checkpoint locally and running the compiled benchmark in the local WSL environment.

INT8 Was Not Prediction-Identical

Dynamic quantization improved speed and size, but it changed some predictions. The final local benchmark showed a small accuracy drop from 90.48% to 89.68%.

This confirms that quantized models must be evaluated carefully.

---

9. Main Findings

The main findings are:

1. **The FP32 baseline provided the best accuracy reference.**
   It achieved 90.48% validation accuracy.
2. **Dynamic INT8 quantization gave the strongest deployment-style improvement.**
   It reduced model size by 48.2% and improved batch-8 latency by 1.83×.
3. **Quantization introduced a small accuracy tradeoff.**
   INT8 accuracy dropped from 90.48% to 89.68%.
4. **torch.compile preserved accuracy but had modest speedup.**
   It improved latency from 435.28 ms to 394.05 ms, but first-call compile overhead was high.

5. **Profiler inspection made the compile result explainable.**
   torch.compile reduced overhead but did not remove the main Linear-layer matrix multiplication bottleneck.

---

10. Conclusion

This project studied efficient transformer inference for text classification using DistilBERT and SST-2. We fine-tuned a full-data FP32 DistilBERT baseline on 67,349 SST-2 training examples and evaluated it on 872 labeled validation examples.

The FP32 baseline achieved:

90.48% validation accuracy
789 / 872 correct

We then compared the baseline with dynamic INT8 quantization and compiled FP32 execution.

Dynamic quantization gave the best practical efficiency improvement:

Latency: 435.28 ms → 238.40 ms
Model size: 255.45 MB → 132.29 MB
Speedup: 1.83×
Size reduction: 48.2%

However, it caused a small accuracy drop:

90.48% → 89.68%

torch.compile preserved accuracy and improved latency slightly:

435.28 ms → 394.05 ms

but first-call compilation overhead was high:

81.52 seconds

The final conclusion is that dynamic quantization was the strongest size and latency optimization for this CPU inference setup, while torch.compile was useful for understanding compiled graph execution but less impactful for this model and environment. The project showed that inference optimization must be evaluated using accuracy, latency, throughput, model size, and prediction behavior together.

---

11. Future Work

Future improvements include:

1. **Run GPU quantization experiments**
   Test GPU-oriented quantization using TensorRT, ONNX Runtime GPU, TensorRT-LLM, or other deployment frameworks.
2. **Tune fine-tuning hyperparameters**
   Try more epochs, learning-rate schedules, early stopping, and larger batch sizes to improve FP32 baseline accuracy.
3. **Evaluate more batch sizes in one consistent final environment**
   The final report focuses on batch size 8. Future work could produce a full consistent table for batch sizes 1, 8, 16, and 32 on the same machine.
4. **Try static quantization or quantization-aware training**
   Dynamic quantization is simple, but other quantization methods may preserve accuracy better.
5. **Build a small demo interface**
   Create a simple command-line or web demo where users enter a sentence and see FP32, INT8, and compiled predictions with latency.
6. **Compare against larger models**
   Test BERT-base, RoBERTa, or newer transformer encoders to see whether optimization effects change with model size.

---

12. GitHub Repository Information

Repository:
https://github.com/amacharla15/CSCI611_GROUP7/tree/main/FinalProject

Project folder:
DistilBERT-TextClassification_Quantization_CompiledExecution/efficient-transformer-inference

Expected repository contents:

src/
  training scripts
  benchmarking scripts
  quantization scripts
  torch.compile scripts
  diagnostic/profiler scripts

results/
  final result CSV files
  benchmark logs
  diagnostic summaries

figures/
  latency charts
  throughput charts
  model size charts
  accuracy/correct-count charts

checkpoints/
  trained model checkpoint or transfer instructions

README.md
PROJECT_PHASE_LOG.md
final presentation file
final report PDF

---

13. References

1. Hugging Face Transformers documentation
   https://huggingface.co/docs/transformers
2. Hugging Face Datasets documentation
   https://huggingface.co/docs/datasets
3. GLUE benchmark / SST-2 dataset
   https://gluebenchmark.com/
4. DistilBERT model
   https://huggingface.co/distilbert-base-uncased
5. PyTorch dynamic quantization documentation
   https://pytorch.org/docs/stable/quantization.html
6. PyTorch torch.compile documentation
   https://pytorch.org/docs/stable/generated/torch.compile.html

7. PyTorch profiler documentation
   https://pytorch.org/docs/stable/profiler.html
8. BERT paper: Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”
   https://arxiv.org/abs/1810.04805
9. DistilBERT paper: Sanh et al., “DistilBERT, a distilled version of BERT”
   https://arxiv.org/abs/1910.01108

---

14. Team Contributions

Abinesh — Dataset, Preprocessing, and Baseline Setup

Abinesh focused on the dataset and baseline setup. This included understanding SST-2, explaining the binary sentiment classification task, describing tokenization, input IDs, attention masks, validation examples, and the baseline modeling pipeline.

Akshith — Optimization and Benchmarking

Akshith focused on implementing and debugging the main optimization workflow. This included full-SST-2 fine-tuning setup, dynamic quantization, torch.compile, benchmarking scripts, accuracy/latency/throughput measurement, profiler inspection, and result correction after moving from the early 4,000-example run to the full 67,349-example run.

Arushi — Results, Visualization, and Interpretation

Arushi focused on analyzing and presenting the experimental results. This included result tables, charts, interpretation of latency/throughput/model size/accuracy tradeoffs, quantization diagnostic explanation, and final conclusion.

Shared Responsibilities

All team members contributed to reviewing the project flow, understanding the final results, preparing the presentation, and making sure the final report and GitHub submission are complete.